

Lab #1: System Call Implementation

Team Members:

- **Manoj Manjunatha : 862481448**
- **Vijay Ram Enaganti : 862545736**
- **Subhash Bangalore Sateesha : 862541575**

Demo video

- link: <https://youtu.be/9XVeBHCSO44?si=2PYbJ5lml1ZE2oKG>

List of all files modified

- Kernel
 - defs.h
 - kalloc.c
 - proc.c
 - proc.h
 - syscall.c
 - syscall.h
 - sysproc.c
- User
 - user.h
 - usys.pl
 - lab1_test.c
- Makefile

Changes Explained

- syscall.h

Here, we define new syscall numbers for `sysinfo` and `procinfo`, which are used to invoke the associated system calls.

```

kernel > C syscall.h > ...
14 #define SYS_sleep 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_hello 22 //hello
24
25 /*****
26
27 #define SYS_sysinfo 23
28 #define SYS_procinfo 24
29
30 *****/
31

```

- syscall.c

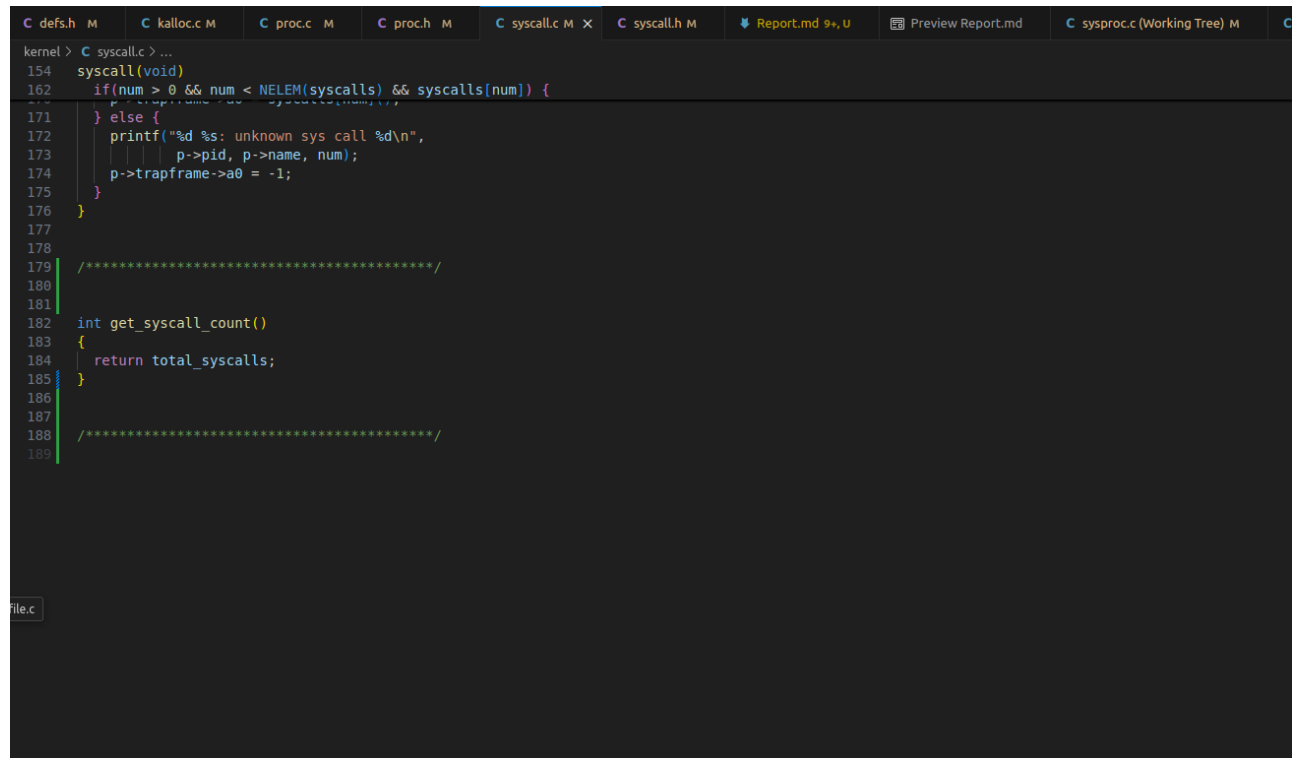
In this section, the system call table is defined. We add an entry to the table, mapping the system call number to its corresponding function.

```

C syscall.c M x C syscall.h M Report.md report 9+,U C syscall.c (Working Tree) M C sysproc.c M C user.h M usys.pl M Report.md 9+
kernel > C syscall.c > ...
116 static uint64 (*syscalls[])(void) = {
138
139 /*****/
140
141 [SYS_hello] sys_hello, //hello: syscall entry
142 [SYS_sysinfo] sys_sysinfo,
143 [SYS_procinfo] sys_procinfo,
144
145 /*****/
146
147 };
148
149
150
151 static int total_syscalls = 0; // added for syscall count
152
153 void
154 syscall(void)
155 {
156 int num;
157 struct proc *p = myproc();
158
159
160
161 num = p->trapframe->a7;
162 if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
163 // Use num to lookup the system call function for num, call it,
164 // and store its return value in p->trapframe->a0
165
166 if(num != SYS_sysinfo){ // added for syscall count
167 total_syscalls++;
168 }
169
170 p->trapframe->a0 = syscalls[num]();
171 } else {
172 printf("%d %s: unknown sys call %d\n",
173 p->pid, p->name, num);
174 p->trapframe->a0 = -1;
175 }
176 }
177
178
179 /*****/
180

```

Additionally, a `total_syscall` variable is introduced to track the total number of system calls made. We have also implemented a `get_syscall_count()` function, which returns the total count of system calls. This helps in monitoring system call usage.

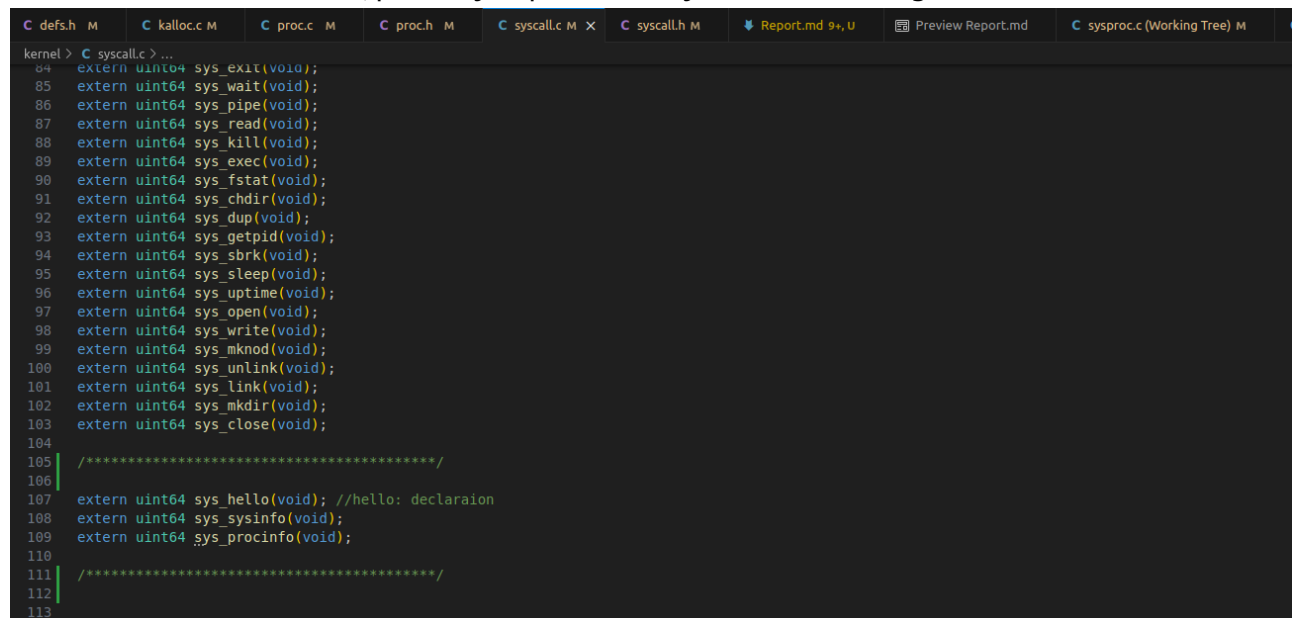


```

kernel > C syscalls.c > ...
154 syscall(void)
162 if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
163     // ...
171 } else {
172     printf("%d %s: unknown sys call %d\n",
173           p->pid, p->name, num);
174     p->trapframe->a0 = -1;
175 }
176 }
177
178
179 /*****/
180
181
182 int get_syscall_count()
183 {
184     return total_syscalls;
185 }
186
187
188 /*****/
189

```

The functions (`sys_hello`, `sys_sysinfo`, and `sys_procinfo`) that return a 64-bit unsigned integer and are defined somewhere else in the code. The `extern` keyword tells the compiler that these functions exist and will be linked later, probably as part of the system call handling in the kernel.



```

kernel > C sysproc.c > ...
84 extern uint64 sys_exit(void);
85 extern uint64 sys_wait(void);
86 extern uint64 sys_pipe(void);
87 extern uint64 sys_read(void);
88 extern uint64 sys_kill(void);
89 extern uint64 sys_exec(void);
90 extern uint64 sys_fstat(void);
91 extern uint64 sys_chdir(void);
92 extern uint64 sys_dup(void);
93 extern uint64 sys_getpid(void);
94 extern uint64 sys_sbrk(void);
95 extern uint64 sys_sleep(void);
96 extern uint64 sys_uptime(void);
97 extern uint64 sys_open(void);
98 extern uint64 sys_write(void);
99 extern uint64 sys_mknod(void);
100 extern uint64 sys_unlink(void);
101 extern uint64 sys_link(void);
102 extern uint64 sys_mkdir(void);
103 extern uint64 sys_close(void);
104
105 /*****/
106
107 extern uint64 sys_hello(void); //hello: declaraion
108 extern uint64 sys_sysinfo(void);
109 extern uint64 sys_procinfo(void);
110
111 /*****/
112
113

```

- `sysproc.c`

Here, system call functions are defined and mapped to their respective system call numbers.

`sys_sysinfo(void)`

This system call retrieves system information based on an integer parameter passed from user space using `argint(0, ¶m)`. The returned information depends on the parameter value:

- `0`: Returns the number of active processes.
- `1`: Returns the total number of system calls made.
- `2`: Returns the number of free memory pages.

A `switch` statement is used to determine which kernel function to invoke. If the parameter is invalid, the function returns `-1`.

`sys_procinfo(struct pinfo *in)`

This system call accepts a pointer to a user-defined `struct pinfo` from user space using `argaddr()`. It populates the structure with the following details about the current process:

- The parent process ID (PID).
- The total number of system calls made by the process.
- The number of memory pages used by the process.

The `copyout()` function is used to safely transfer kernel data to the user-provided memory space.

```
kernel > C sysproc.c > sys_sysinfo(void)
93
94 /*****/
95
96
97 uint64 sys_hello(void) // hello syscall definition
98 {
99     int n;
100     argint(0,&n);
101     print_hello(n);
102     return 0;
103 }
104
105 uint64
106 sys_sysinfo(void)
107 {
108     int param;
109     argint(0,&param);
110
111     switch (param) {
112         case 0:
113             return count_active_processes();
114         case 1:
115             return get_syscall_count();
116         case 2:
117             return count_free_pages();
118         default:
119             return -1;
120     }
121 }
122
123 uint64 sys_procinfo(struct pinfo *in)
124 {
125     argaddr(0,(void*) &in);
126     if(!in)
127         return -1;
128
129     struct proc *p = myproc();
130     copyout(p->pagetable, (uint64)&in->ppid,(char *)&p->parent->pid, sizeof(p->pid));
131     int N_pages = p->sz / PGSIZE;
132
133     copyout(p->pagetable,(uint64)&in->syscall_count,(char *)&p->syscall_count, sizeof(int));
134     copyout(p->pagetable,(uint64)&in->page_usage,(char *)&N_pages, sizeof(int));
135     return 0;
136 }
137
138 /*****/
139
```

- `proc.c` Kernel Function Declarations

`int count_active_processes(void)`

This function iterates through the global process table (`proc` array) and counts the number of active processes, processes that are not in the `UNUSED` state. It returns the total count of processes that are either running, sleeping, or in any state other than `UNUSED`.

- Loop through all entries in the `proc` array.
- If a process is not in the `UNUSED` state, increment the count.
- Return the final count.

`int procinfo(struct pinfo *in)`

This function populates a `struct pinfo` in user space with details about the calling process (`myproc()`), including:

- The parent process ID.
- The total number of system calls made by the process.
- The number of memory pages used by the process.

It uses `copyout()` to safely transfer kernel data to the user-provided memory, ensuring secure access for the user program.

- Validate the input pointer.
- Retrieve the current process using `myproc()`.
- Calculate the number of memory pages as `sz / PGSIZE`.
- Populate the `struct pinfo` with the parent PID, system call count, and memory page usage.
- Use `copyout()` to transfer the data to user space.

```

682 }
683 }
684
685 /*****
686
687
688 void print_hello(int n)
689 {
690     printf("Hello from the kernel space %d\n",n);
691 }
692
693
694 int count_active_processes(void) {
695     int count = 0;
696
697     struct proc *p;
698
699     for(p = proc; p < &proc[NPROC]; p++){
700         if(p->state != UNUSED)
701             count++;
702     }
703     return count;
704 }
705
706
707
708
709 int procinfo(struct pinfo *in)
710 {
711     if(!in)
712         return -1;
713
714     struct proc *p = myproc();
715     copyout(p->pagetable, (uint64)&in->ppid, (char *)&p->parent->pid, sizeof(p->pid));
716     int N_pages = p->sz / PGSIZE;
717
718     copyout(p->pagetable, (uint64)&in->syscall_count, (char *)&p->syscall_count, sizeof(int));
719     copyout(p->pagetable, (uint64)&in->page_usage, (char *)&N_pages, sizeof(int));
720     return 0;
721 }
722
723
724
725 /*****
726

```

- proc.h

In this file, we have declared the `struct pinfo`, which is used to store and manage process-related information. This structure used in implementing the `procinfo` system call.

```

kernel > C proc.h > ...
85 struct proc {
86     struct proc *parent;          // Parent process
87
88     // these are private to the process, so p->lock need not be held.
89     uint64 kstack;                // Virtual address of kernel stack
90     uint64 sz;                    // Size of process memory (bytes)
91     pagetable_t pagetable;        // User page table
92     struct trapframe *trapframe;  // data page for trampoline.S
93     struct context context;        // switch() here to run process
94     struct file *ofile[NOFILE];   // Open files
95     struct inode *cwd;             // Current directory
96     char name[16];                // Process name (debugging)
97
98     int syscall_count;            // syscall count      // added
99 };
100
101
102
103
104
105
106
107
108
109
110
111
112
113 /*****
114
115 struct pinfo {
116     int ppid;
117     int syscall_count;
118     int page_usage;
119 };
120
121
122 /*****
123
  
```

- kalloc.c

The `count_free_pages()` function traverses the linked list of free memory pages (`kmem.free_list`) to count how many pages are currently available for allocation. It first acquires a lock to ensure no other process modifies the list during traversal, then iterates through each node (`struct run *r`), incrementing `free_count` for every free page found, and finally releases the lock before returning the total count.

```

kernel > C kalloc.c > ...
85
86  /***/
87
88  int count_free_pages()
89  {
90
91      struct run *r;
92      int free_count = 0;
93      acquire(&kmem.lock); // Lock the kmem structure.
94      r = kmem.freelist;
95      while (r)
96      {
97          free_count++;
98          r = r->next;
99      }
100
101      release(&kmem.lock);
102      return free_count;
103  }
104
105
106
107  /***/
108

```

- defs.h

In this file, we declare the function prototypes that are implemented in `sysproc.c`. These declarations ensure that the kernel can correctly reference and invoke the corresponding system call functions.

```

kernel > C defs.h > ...
187
188
189  /***/
190  void      print_hello(int); //hello
191
192  int      count_active_processes(void);
193  int      get_syscall_count(void);
194  int      count_free_pages(void);
195  /***/
196
197
198  // number of elements in fixed-size array
199  #define NELEM(x) (sizeof(x)/sizeof((x)[0]))
200

```

- usys.pl

In this file, we add the new system calls `sysinfo` and `procinfo` at the user level. These additions ensure that the system calls are accessible from user programs.

```

user > usys.pl
27 entry("open");
28 entry("mknod");
29 entry("unlink");
30 entry("fstat");
31 entry("link");
32 entry("mkdir");
33 entry("chdir");
34 entry("dup");
35 entry("getpid");
36 entry("sbrk");
37 entry("sleep");
38 entry("uptime");
39
40
41
42 entry("hello");
43 entry("sysinfo");
44 entry("procinfo");
45

```

- user.h

In this file, we declare the `sysinfo` and `procinfo` system calls at the user level. These declarations specify the return types and arguments for the system calls, enabling their usage in user programs.

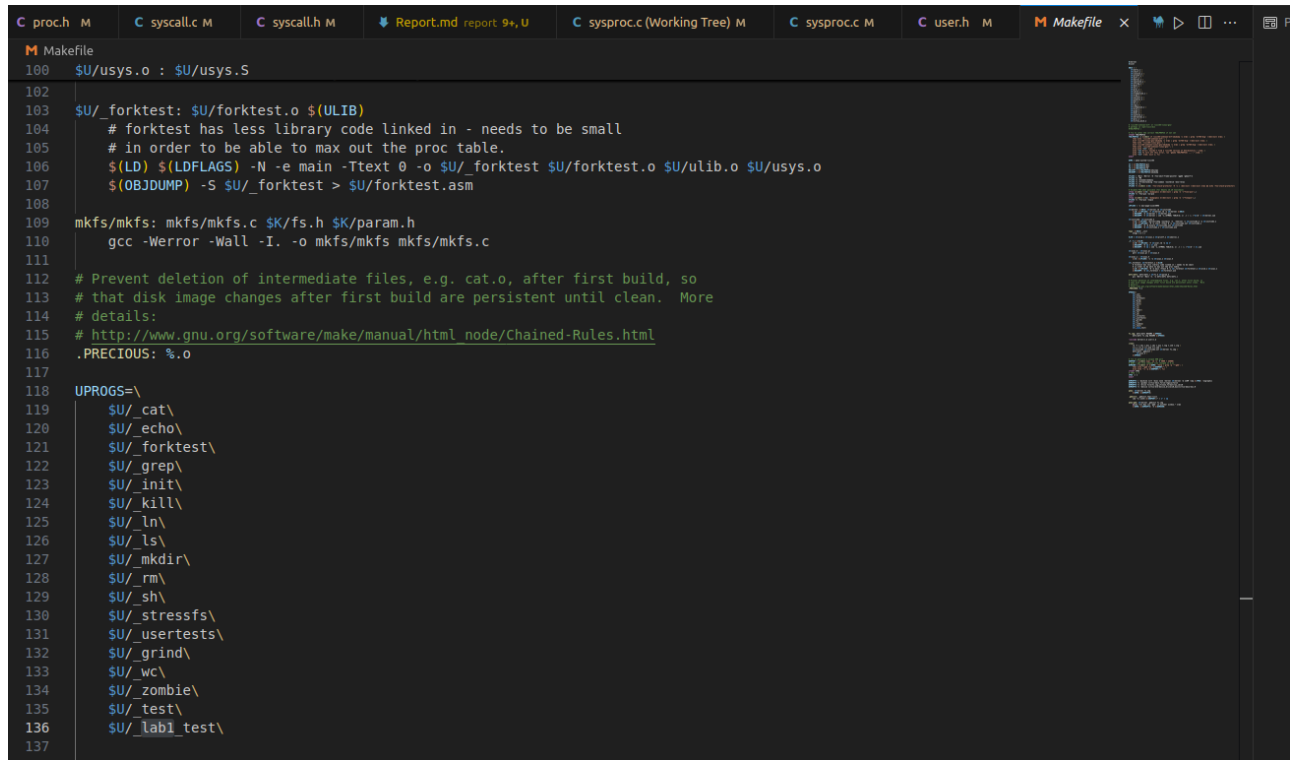
```

user > C user.h > ...
11 int kill(int);
12 int exec(const char*, char**);
13 int open(const char*, int);
14 int mknod(const char*, short, short);
15 int unlink(const char*);
16 int fstat(int fd, struct stat*);
17 int link(const char*, const char*);
18 int mkdir(const char*);
19 int chdir(const char*);
20 int dup(int);
21 int getpid(void);
22 char* sbrk(int);
23 int sleep(int);
24 int uptime(void);
25
26 /*****
27
28 int hello(int); //hello
29 int sysinfo(int);
30 int procinfo(struct pinfo*); // syscall to get process info
31
32 /*****
33
34

```

- Makefile

In the `Makefile`, we added `lab1_test` to the list of executables.



```

100 $U/usys.o : $U/usys.S
102
103 $U/_forktest.o : $U/_forktest.o $(ULIB)
104 # forktest has less library code linked in - needs to be small
105 # in order to be able to max out the proc table.
106 $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/_forktest.o $U/_lib.o $U/usys.o
107 $(OBJDUMP) -S $U/_forktest > $U/_forktest.asm
108
109 mkfs/mkfs: mkfs/mkfs.c $K/fs.h $K/param.h
110 gcc -Werror -Wall -I. -o mkfs/mkfs mkfs/mkfs.c
111
112 # Prevent deletion of intermediate files, e.g. cat.o, after first build, so
113 # that disk image changes after first build are persistent until clean. More
114 # details:
115 # http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
116 .PRECIOUS: %.o
117
118 UPROGS=\
119 $U/_cat\
120 $U/_echo\
121 $U/_forktest\
122 $U/_grep\
123 $U/_init\
124 $U/_kill\
125 $U/_ln\
126 $U/_ls\
127 $U/_mkdir\
128 $U/_rm\
129 $U/_sh\
130 $U/_stressfs\
131 $U/_usertests\
132 $U/_grind\
133 $U/_wc\
134 $U/_zombie\
135 $U/_test\
136 $U/_lab1_test\
137

```

Steps Involved

1. Define the System Call Numbers

Add the system call numbers for `sysinfo` and `procinfo` in `syscall.h`.

2. Update the System Call Table

Add entries for the `sysinfo` and `procinfo` system calls in the system call table located in `syscall.c`.

3. Implement the System Call Handlers

Write the handler functions for `sysinfo` and `procinfo` in `sysproc.c`.

- **sysinfo**: Handles requests to retrieve system-wide information such as:
 - Number of active processes.
 - Number of free memory pages.
 - Total number of system calls made.
- **procinfo**: Handles requests to retrieve information about a specific process, such as:
 - Process ID (PID).
 - Parent Process ID (PPID).
 - Process state.
 - Memory usage.

4. Implement Helper Functions

Create helper functions to:

- Count active processes.
- Count free memory pages.
- Track the total number of system calls.

- Retrieve process-specific information for `procinfo`.

5. Add User-Level Declarations

Declare the `sysinfo` and `procinfo` system calls in `user.h`.

6. Update `usys.pl`

Add entries for the `sysinfo` and `procinfo` system calls in `usys.pl` to make them accessible at the user level.

7. Build and Test

Compile the code and run test programs to verify the implementation of both system calls.

- **Test for `sysinfo`:** Write a program to display system-wide information such as the number of active processes, free memory pages, and total system calls.
- **Test for `procinfo`:** Write a program to display information about a specific process by passing its PID as an argument.

How the `info` system call is processed, from the user-level program into the kernel code, and then back into the user-level program.

a. SYSINFO:

The `sysinfo` system call takes an integer argument from user space and returns a value based on the argument provided. At the user level, the `sysinfo()` function is called with a specific parameter. In the kernel (specifically in `sysproc.c`), this argument is retrieved using the built-in `argint()` function. Based on the value received, a switch statement determines which action to perform, such as returning the number of active processes, the system call count, or the number of free memory pages. The result is then returned to the user-level program (`lab1_test.c`), where it can be printed or used further. This is the high-level flow of the `sysinfo` system call.

b. PROCINFO:

The `procinfo` system call takes a pointer to a structure (defined in user-level code like `lab1_test.c`) and returns 0 on success or -1 on failure. At the user level, `procinfo()` is called with the address of a struct `pinfo` variable. In the kernel, the pointer is retrieved using the `argaddr()` function. The system then accesses the current process's PCB to gather the parent process ID, the number of system calls made, and the amount of memory used (in pages). These values are written back to user space using the `copyout()` function, which safely copies data from kernel space to user space. After successfully copying all the required information, the function returns 0. Back in `lab1_test.c`, the user program reads the values from the struct and prints the results. This is the overall logic and flow of how the `procinfo` system call works.

Contributions

- **Manoj Manjunatha:**

- Implemented the `sysinfo` system call, including its handler function `get_syscall_count` and `count_active_processes`.
- Updated the system call table and defined system call numbers in `syscall.h` and `syscall.c`.
- Contributed to writing the report.

- **Subhash Bangalore Sateesha:**

- Implemented the `count_free_pages()` function.
- Modified `usys.pl` to generate assembly stubs for both `sysinfo` and `procinfo`.
- Added the user-level declaration and assembly stub for `procinfo`.
- Updated the `Makefile` to include the test program `lab1_test.c`.

- **Vijay Ram Enaganti:**

- Implemented the `procinfo` system call, including its handler and helper functions.
- Added the test program `lab1_test.c` to verify the functionality of `procinfo`.
- Contributed to writing the report.